Kapitel 4

Systementwurf & Softwarearchitektur

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Agenda

Abschnitt 4.1

Einführung zu Systementwurf & Softwarearchitektur

Abschnitt 4.2

Elemente & Dokumentation einer Softwarearchitektur

Abschnitt 4.3

Architekturmuster

Klausurrelevante Lernziele

- Wesentliche Ziele einer Softwarearchitektur erläutern können
- Baustein, Schnittstelle und deren Zusammenwirken erläutern können
- Phasen, Modelle & wesentliche Schritte des Object-Oriented Software Engineering (OOSE) nennen können
- Unterschied zwischen Architekturmuster, Entwurfsmuster & Idiom erläutern können
- Architekturmuster Layer & Model-View-Controller (MVC) erläutern sowie einige Vor- bzw. Nachteile nennen können
- Elemente der Drei-Schichten-Architektur kennen

Option: Zertifizierung

CPSA-F®
Certified Professional for Software Architecture
Foundation Level

- Certified Professional for Software Architecture® (CPSA)
- Weltweit anerkanntes Weiterbildungsschema für Softwarearchitekturen
  - Durch: International Software Architecture Qualification Board (iSAQB)
  - Level: Foundation, Advanced, Expert
- Hier: Foundation Level (CPSA-F)
  - Aktuelles Curriculum: Version 2025.1
- Prüfung bei unabhängiger Zertifizierungsstelle
  - Multiple Choice, ca. 40 Fragen, 75 Minuten, Bestehensquote: 60%, deutsch
  - Gebühr: ca. 265 € (50% Studierendenrabatt möglich)
  - Zertifikat ist lebenslang gültig
  - Online-Prüfung möglich
- Vorlesung bereitet teilweise auf die Prüfung vor
- Tiefergehende Behandlung im Modul »Softwaretechnik II« im Master »Angewandte Mathematik und Informatik«

Abschnitt 4.1

Einführung zu Systementwurf & Softwarearchitektur

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Ziel: langlebige Softwarearchitekturen

- »Verrottende Systeme« mit wachsenden technischen Schulden
  - Wartungs- und Änderungskosten steigen unaufhaltsam
  - Niemand überblickt die Konsequenzen von Änderungen; selbst kleinste Erweiterungen sind enorm risikoreich
  - Die Arbeit mit dem System wird zur Qual, die Einarbeitung neuer Mitarbeiter ist sehr aufwändig
  - CRAP Cycle: Create → Repair → Abandon → rePlace
- Achtung: stetige »Erosion« auch bei gut entworfenen Systemen
- Gegenmittel
  - Aktive Softwarearchitektur; Einsatz von Architektur- & Entwurfsmustern
  - Entwurfsprinzipien & Clean Code: Modularisierung, lose Kopplung & hohe Kohäsion [→ Kapitel 5]
  - Laufende Qualitätsmaßnahmen & Refactoring
- Was bedeutet Softwarearchitektur?
  - Innere Struktur eines Softwaresystems: Bausteine & Schnittstellen, Schichten
  - Summe aller wichtigen Entwurfsentscheidungen, die im Verlauf der weiteren Entwicklung nur noch schwer zu ändern sind

Quelle: (Vogel 2015; Kruchten 2003)

Systementwurf & Softwarearchitektur

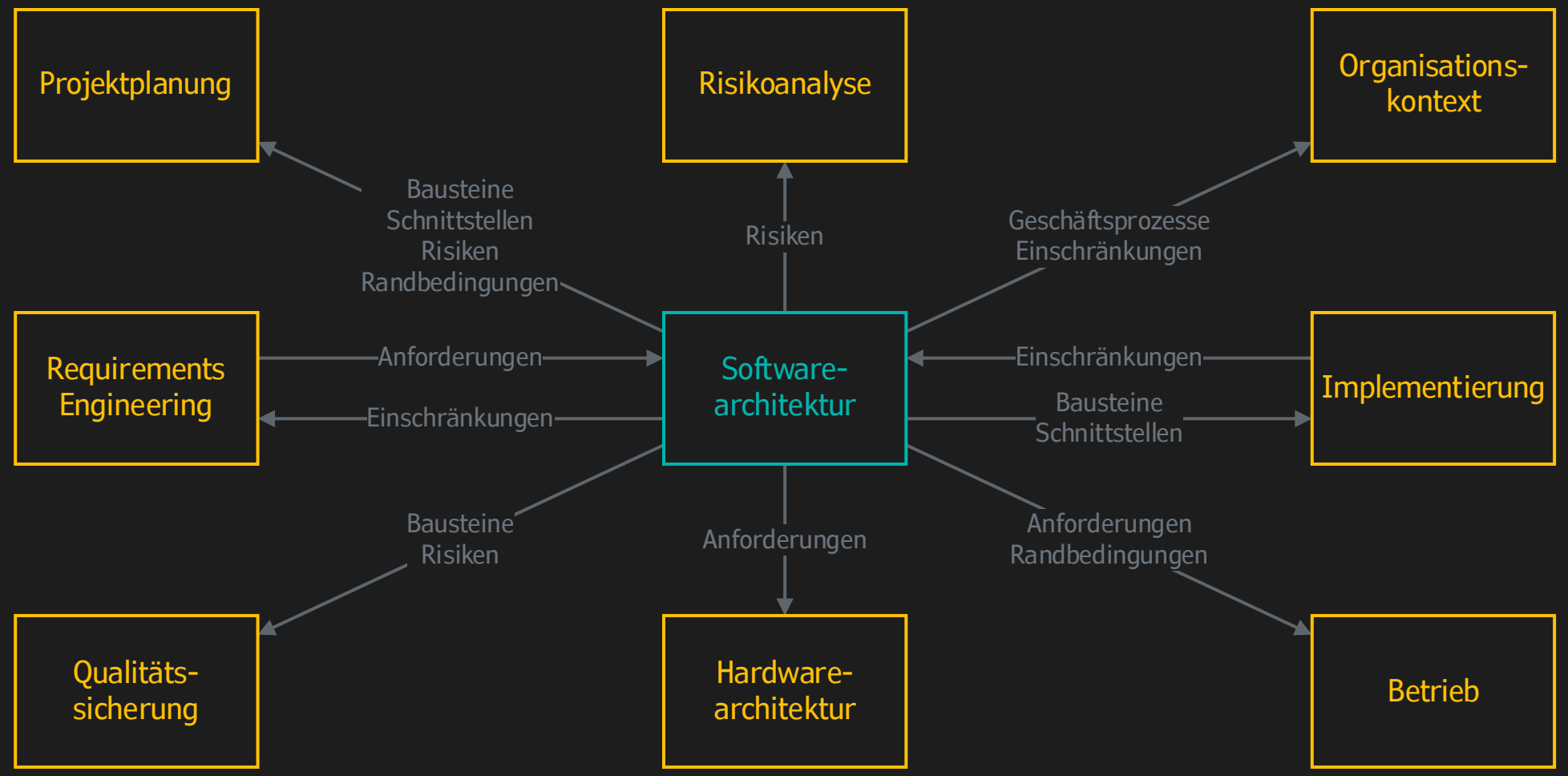
- Phasen: nach der Requirements Analysis und vor der Implementation wird zunächst die Software modelliert und ein Systementwurf erstellt

Design – Process of defining the software architecture, components, modules, interfaces, and data for a software system to satisfy specified requirements.

Architecture –Fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution.

(ISO/IEC/IEEE 24765:2017)

Kontext der Softwarearchitektur



Quelle: nach (Starke 2024)

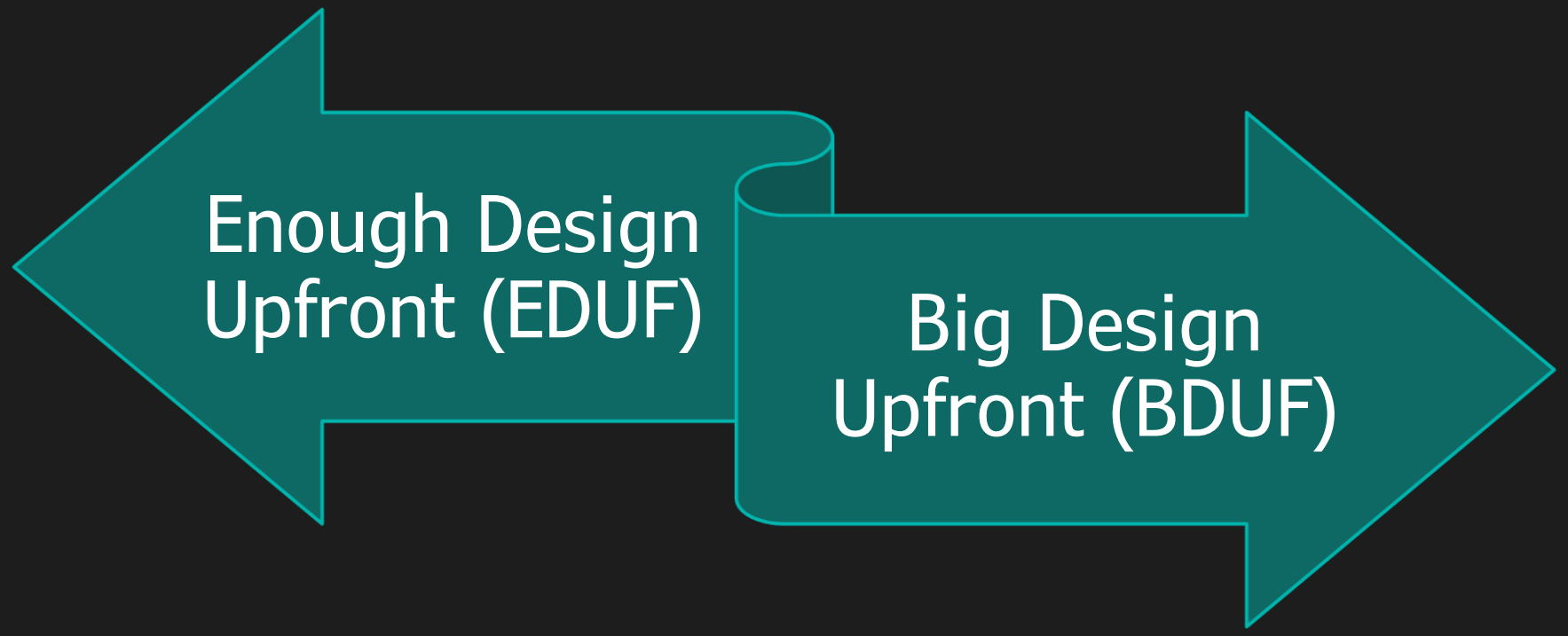
Softwarearchitektur (1)

- Beschreibt Lösungskonzept zur Erfüllung der funktionalen & nicht-funktionalen Anforderungen
  - Soll-Zustand einer neuen Softwaresystems (Engineering), oder
  - Ist-Zustand eines bestehenden Softwaresystems (Reengineering)
- Basiert auf dokumentieren Entwurfsentscheidungen
- Ist technologieneutral & unabhängig vom Vorgehensmodell
- Liefert das »Big Picture«, schafft Qualität & stellt Erweiterbarkeit sicher
- Gibt Orientierung im gesamten Lebenszyklus einer Software
  - Welche Komponenten werden zugeliefert / eingekauft / entwickelt?
  - Welche Komponenten sind entwickelt / getestet / in Bearbeitung?
  - Wie kann die Software sinnvoll getestet werden?
  - Wie wird die Software installiert bzw. verteilt? Wie gewartet?
  - Wie kann eine Fehlerquelle eingegrenzt werden (Debugging)?
  - Welche Teile wären von Änderungen betroffen (Impact Analysis)?

Softwarearchitektur (2)

- Liefert den fachlichen & technischen Kontext
  - Umgebung, Nachbarsysteme, Nutzungsgruppen
  - Schnittstellen zu externen Systemen, die genutzt bzw. angeboten werden müssen; Kommunikationsformen
  - Stabilität, Performanz, Anpassbarkeit der Schnittstellen
- Beschreibt technische Einflussfaktoren
  - Hardware- und Netzwerkinfrastruktur
  - Softwareinfrastruktur: Betriebssysteme, Datenbanksysteme, Middleware, Webserver, Verzeichnisdienste
  - Bibliotheken, Frameworks, Components off the Shelf (COTS)
  - Programmiersprachen, Datenstrukturen, Architekturmuster, Entwurfsmuster, Coding Conventions, Referenzarchitekturen
- Beschreibt organisatorische Einflussfaktoren
  - Analyse- und Entwurfsmethoden, Vorgehensmodell, Dokumentation
  - Entwicklungsumgebung, Konfigurationsmanagement, Testprozesse
  - Qualitätsstandards; vertragliche, juristische & regulatorische Faktoren

Wie viel Softwarearchitektur?



- BDUF: Annahme, dass zu Beginn alle Anforderungen genau beschrieben werden können und sich nicht mehr ändern [→ Wasserfallmodell]
- EDUF: Respektiert, dass Anforderungen noch nicht alle exakt bekannt und Änderungen normal sind [→ agile Modelle]
- Nicht empfohlen: No Design Upfront (NDUF)

Kategorien von Softwaresystemen

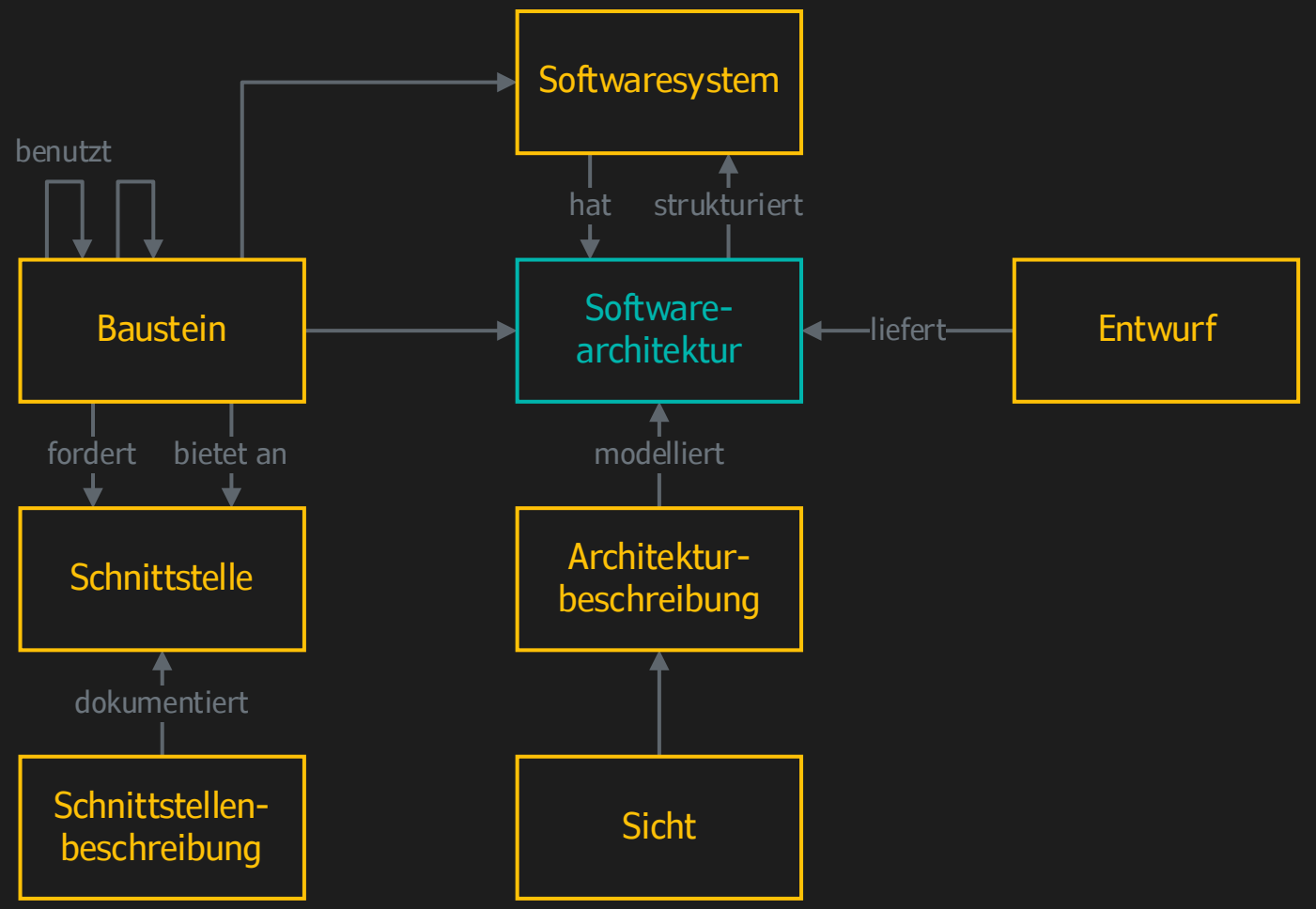
- Kategorisierung eines Softwaresystems hilft bei der Erstellung einer Softwarearchitektur, z.B. durch Architekturmuster
- Keine einheitliche Kategorisierung von Software
  - Anwendungs-, Unterstützungs-, Systemsoftware gemäß (IEEE 610.12-1990)
  - Standard- versus Individualsoftware
  - Anwendungsdomäne: Versicherungen, Medizin, Produktion etc.
- Hier: Enterprise Systems; im Consumer-Bereich weitere Kategorien, z.B. Spiele
- Beispiele
  - Hintergrundsysteme, z.B. Offline- oder Batch-Systeme
  - Decision Support Systems, z.B. Business Intelligence, Data Warehouse
  - Interaktive Online-Systeme, z.B. betriebliche Informationssysteme
  - Webanwendungen & mobile Apps
  - Eingebettete Systeme (Embedded Systems)
  - Systeme mit Echtzeitanforderungen (Real-Time Systems)

Abschnitt 4.2

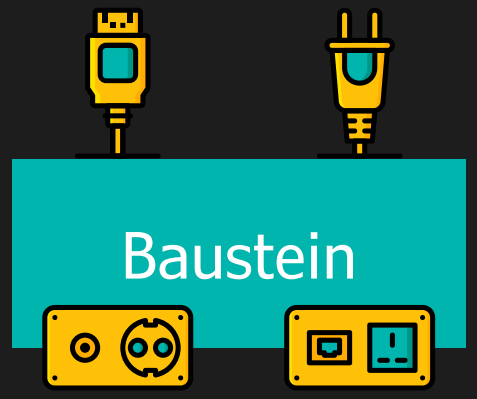
Elemente & Dokumentation einer Softwarearchitektur

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Elemente einer Softwarearchitektur

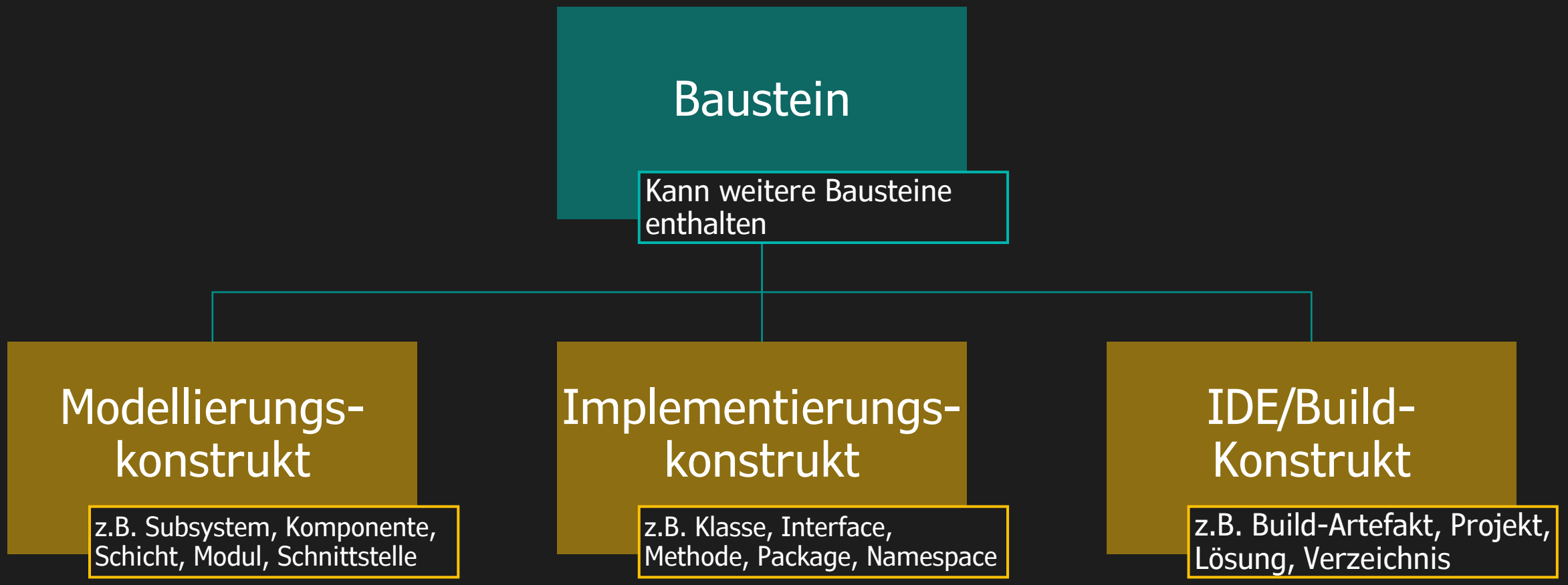


Bausteine & Schnittstellen



- Baustein
  - Exportiert & importiert Schnittstellen
  - Garantiert zugesicherte Funktionalität
  - Kapselt die Implementierung
  - Kann durch anderen Baustein mit gleichen Schnittstellen ersetzt werden
- Schnittstelle
  - Repräsentiert definierten Zugangspunkt zum Softwaresystem
  - Garantiert Eigenschaften des Zugangspunkts, z.B. Attribute, Funktionen [→ Design by Contract]
  - Wird von Consumer (aka Importeur, Client, Dienstnehmer) importiert
  - Wird von Provider (aka Exporteur, Server, Dienstanbieter) exportiert
- Hierarchische Komposition
  - Baustein enthält ggf. Unterbausteine & kapselt diese
  - Äußere Schnittstellen können auf innere delegieren werden
  - Auf diese Weise wird hierarchische Beziehung zwischen Bausteinen definiert

Arten von Bausteinen



Quelle: (Starke 2024)

Design by Contract (DbC)

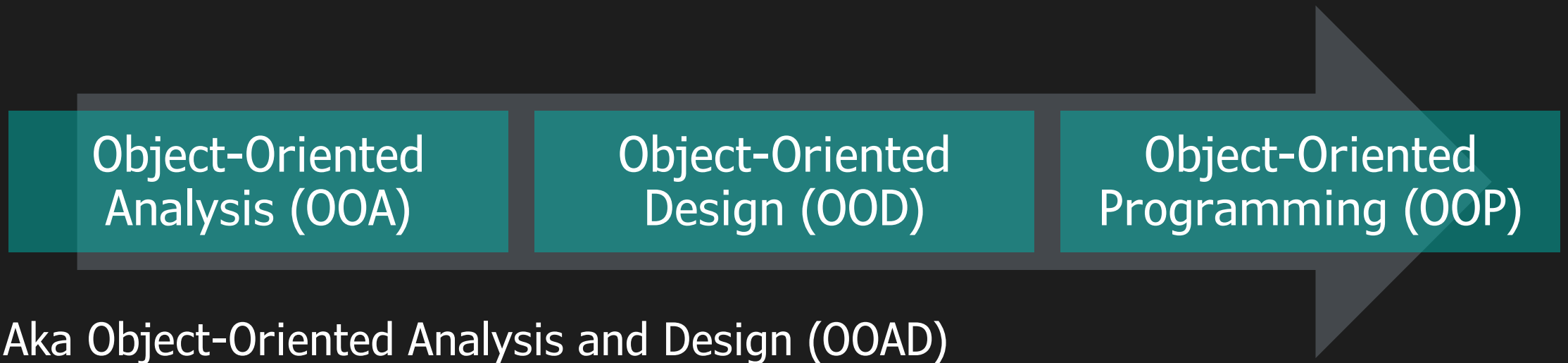
- Auch: Contract Programming, Bertrand Meyer
- Definition formaler »Verträge« für Schnittstellen, um reibungsloses Zusammenspiel der Bausteine zu gewährleisten
- Beispiel: Vertrag für eine Klassenoperation besteht aus
  - Vorbedingungen (Precondition): Zusicherungen, die der Aufrufer der Operation einzuhalten hat
  - Nachbedingungen (Postcondition): Zusicherungen, die die aufgerufene Operation einhalten wird
  - Invarianten (Class Invariants): über alle Instanzen einer Klasse hinweg geltende Annahmen
  - Weitere Elemente: Seiteneffekte (Side Effects), Fehlerbehandlung etc.
- Formale Spezifikation durch Object Contraint Language (OCL)
- Programmiersprachen: nativ (Eiffel), Code Contracts (C#), Bean Validation (Java)
- Zusammenhang mit »Defensiver Programmierung«

Quelle: (Meyer 1986)

Schnittstellen entwerfen

- Provider zur Entwurfszeit oder erst zur Laufzeit bekannt?
- Wer definiert Schnittstelle: Provider? Consumer? Standards? Adapter nötig?
- Typen von Schnittstellen
  - Sync-Call-Return: Consumer ruft Dienst des Providers auf und wartet auf Antwort
  - Async: Consumer ruft Dienst auf und erwartet später eine Antwort
  - Fire & Forget: Consumer benachrichtigt Provider, ggf. Empfangsbestätigung (Acknowledge)
  - Publisher-Subscriber: Verteilung einer Nachricht an Abonnenten
- Ansätze: Application Programming Interface (API), Webservice, RESTful
- Weitere Aspekte
  - Atomare Informationseinheiten & primitive Datentypen
  - Sicherheit: Authentifizierung, Autorisierung, Verschlüsselung etc.
  - Garantierte Performance & Verfügbarkeit (Qualities of Service, QoS)
  - Kosten: Lizenz, Abrechnung pro Aufruf oder Datenvolumen etc.
  - Ggf. Buchhaltungs- und fälschungssichere Protokollierung der Aufrufe
- Eigene Schnittstellen immer selbst benutzen (»Eat Your Own Dogfood«)

Object-Oriented Software Engineering (OOSE)



- Aka Object-Oriented Analysis and Design (OOAD)
- Mitte 1990er: Grady Booch, James Rumbaugh, Ivar Jacobson (Three Amigos) und weitere; zusammen mit UML
- Ziele
  - Software an der Sprache und Denken der Nutzenden ausrichten
  - Verwendung derselben Konzepte & Notation in allen drei Phasen
  - Software soll einfacher erweitert, getestet & gewartet werden können
- OOA-Ergebnis: Analysemodell, Grundlage für OOD
- OOD-Ergebnis: Entwurfsmodell, Grundlage für OOP

OOA & Analysemodell

- Iterativer Prozess auf Basis der Anforderungen aus Requirements Analysis
- Schritt 1: Anwendungsfälle der Domäne ermitteln
- Schritt 2: Relevante statische Strukturen ermitteln
  - Entitäten der Domäne [→ Klassen] identifizieren: Baupläne für Objekte mit gemeinsamer Datenstruktur & gemeinsamem Verhalten
  - Gemeinsame Attribute & Operationen von Klassen identifizieren
  - Zuordnungen von Attributen & Operationen zu Instanzen oder Klasse [→ statische Elemente]
  - Generalisierungen & Spezialisierungen von Klassen identifizieren
  - Beziehungen zwischen Klassen identifizieren
  - Strukturen modellieren [→ UML-Klassendiagramme]
- Schritt 3: Dynamisches Verhalten der Strukturen ermitteln
  - Externes Verhalten modellieren [→ UML-Aktivitäts-, Zustands- und Sequenzdiagramme]

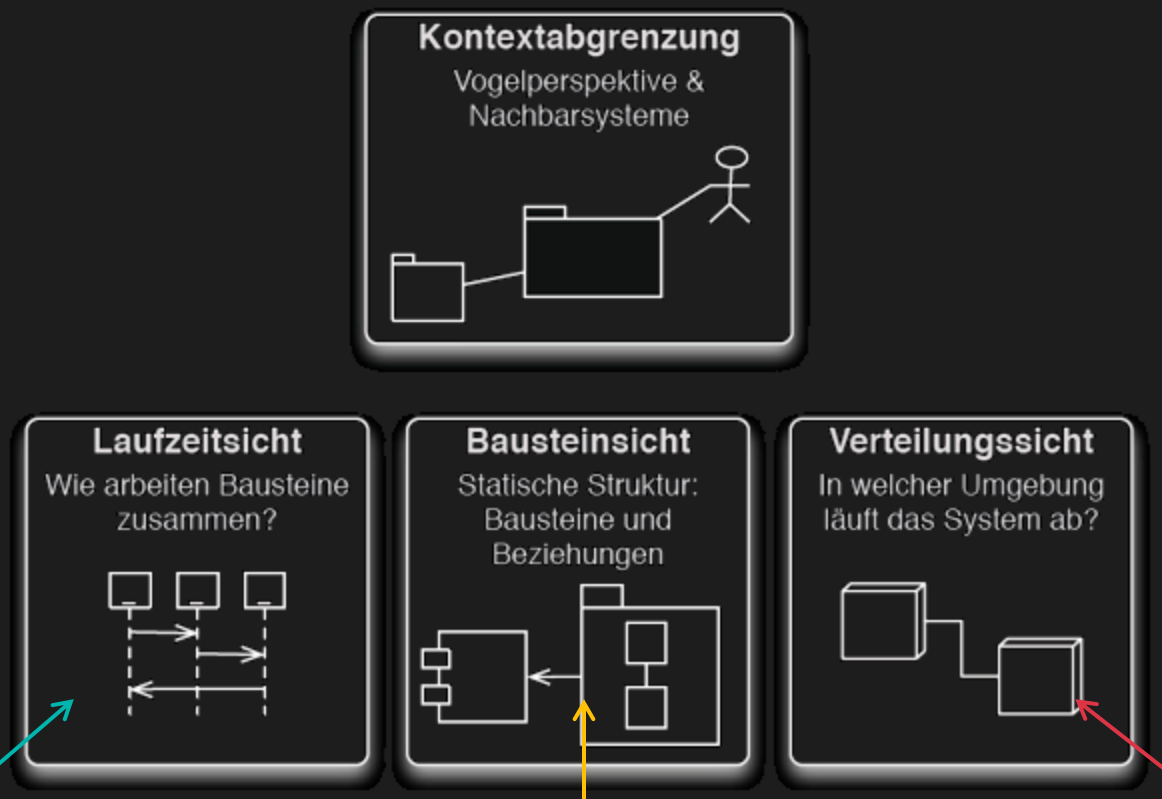
OOD & Entwurfsmodell

- Iterativer Prozess auf Basis des Analysemodells
  - Fachlichen & technischen Kontext, technische & organisatorische Einflussfaktoren, Randbedingungen identifizieren
  - Architekturmuster identifizieren & anwenden
  - Bausteine & Schnittstellen identifizieren [→ UML-Komponentendiagramme]
  - Application Framework definieren: Frameworks, Packete, COTS etc.
  - Analysemodell verfeinern & optimieren [→ UML-Klassen-, Aktivitäts-, Zustands-, Sequenz- und Kommunikationsdiagramme]
  - Bedienungsoberfläche entwerfen
  - Datenmodelle entwerfen & Persistierung von Daten planen
  - Entwurfsprinzipien anwenden [→ Kapitel 5]
  - Entwurfsmuster identifizieren & anwenden
  - Deployment & Installation planen [→ UML-Verteilungsdiagramme]
- Ergebnis: Architektur / Entwurfsmodell als Grundlage für Implementierung (OOP)

Dokumentation einer Softwarearchitektur

- Softwarearchitektur muss für alle Stakeholder dokumentiert werden
- Grundregeln
  - Dokumentation: aktuell, korrekt, verständlich und strukturiert
  - Aus Sicht der Stakeholder formulieren & regelmäßig prüfen
  - Annahmen & Rahmenbedingungen zeigen, Entscheidungen begründen
  - Mehrdeutigkeiten & Wiederholungen vermeiden
  - Notation erklären (Glossar), Standards nutzen (z.B. UML)
  - Offene Punkte kennzeichnen
- Verschiedene Vorlagen, z.B. arc42 von Peter Hruschka, Gernot Starke (2005)
  - Creative Common 3.0
  - Aktuelle Version 8 (Feb. 2022)
  - Weite Verbreitung in DACH-Region
  - Vorlagen mit Erläuterungen & Beispiele verfügbar
  - Inhalte u.a.: Randbedingungen, Kontextabgrenzung, Lösungsstrategie, Bausteinsicht, Laufzeitsicht, Verteilungssicht, Architekturentscheidungen, Qualitätsanforderungen, Risiken & technische Schulden, Glossar

arc42: Sichten & UML-Diagramme



Aktivitätsdiagramm
Zustandsdiagramm
Sequenzdiagramm

Komponentendiagramm
Klassendiagramm

Verteilungsdiagramm

Quelle: (Starke 2024)

Abschnitt 4.3

Architekturmuster

FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

Muster in der Softwaretechnik

Architekturmuster

Systemebene (höchste Abstraktionsebene)

Struktur von IT-Systemen als Ganzes

Bausteine, Schnittstellen, Beziehungen & Verantwortlichkeiten

Entwurfsmuster

Klassenebene (mittlere Abstraktionsebene)

Struktur einzelner Klassen bzw. Klassenverbände

Erzeugungsmuster

Strukturmuster

Verhaltensmuster

Idiome

Quelltextebene (niedrigste Abstraktionsebene)

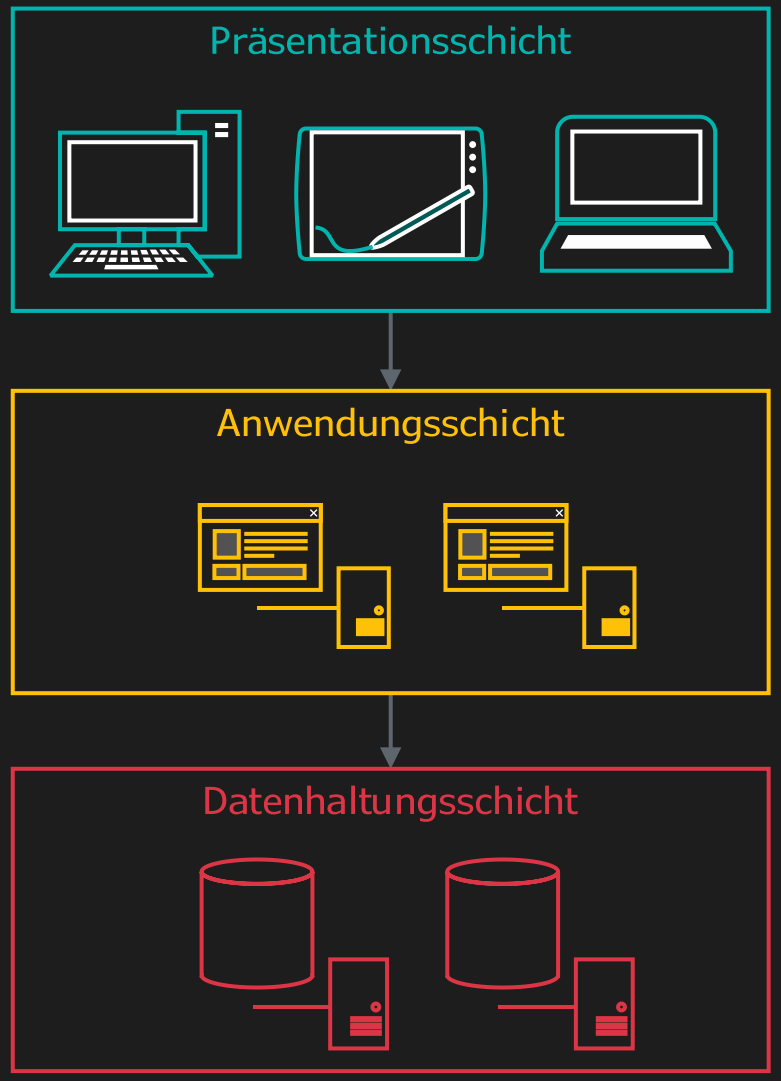
Coding Conventions [→ Kapitel 5]

Architekturmuster: Überblick

- Datenflusssysteme und datenzentrische Systeme
  - Folge von Operationen auf Daten bzw. gemeinsam verwendeter, zentraler Datenbestand
  - Muster: z.B. Batch Sequential, Pipes & Filters, Repository, Blackboard
- Hierarchische Systeme
  - Muster: z.B. Primary & Replicas, Layer (Schichten), Ports & Adapter, virtuelle Maschine
- Interaktionsorientierte Systeme mit grafischer Bedienungsoberfläche (GUI)
  - Muster: z.B. MV*-Modelle wie Model-View-Controller (MVC), Model-View-Presenter (MVP), Model-View-Template (MVT) und Model-View-ViewModel (MVVM)
- Verteilte Systeme und Cloud Computing
  - Unabhängige Bausteine, die über Kommunikationsnetze interagieren
  - Muster: z.B. Client-Server, CQRS, Broker, Peer-to-Peer, Publisher-Subscriber, Message Queue
  - Cloud: zusätzliche Herausforderungen bzgl. Skalierbarkeit, Überwachung, Resilienz & Sicherheit
  - Event-Driven Architecture (EDA)
  - Service-Oriented Architecture (SOA)
  - Resource-Oriented Architecture (ROA)
  - Microservices Architecture (MSA)

Layer (1)

- Schicht (Layer)
  - Für bestimmte Aufgabe zuständig
  - Kapselt Details & Elemente mit ähnlichem Abstraktionsgrad
  - Bietet Dienste der darüberliegenden Schicht an
  - Nutzt Dienste der darunterliegenden Schicht
- Beispiele
  - Kommunikationsprotokolle, z.B. ISO/OSI-Referenzmodell
  - Betriebssysteme
  - Drei-Schichten-Architektur
  - Domain-Driven Design (vier Schichten)
- Unterschied »Layer« versus »Tier«
  - Layer: logische Trennung; können auf einer Maschine ausgeführt werden
  - Tier: physikalische Trennung; verschiedene Maschinen
  - Tiers benötigen Layers, aber nicht umgekehrt



Beispiel: Drei-Schichten-Architektur

Layer (2)

Vorteile

- Verständliche Strukturierung von Abhängigkeiten
- Schichten sind voneinander unabhängig
- Schichten sind austauschbar, wenn sie gleiche Dienste umfassen

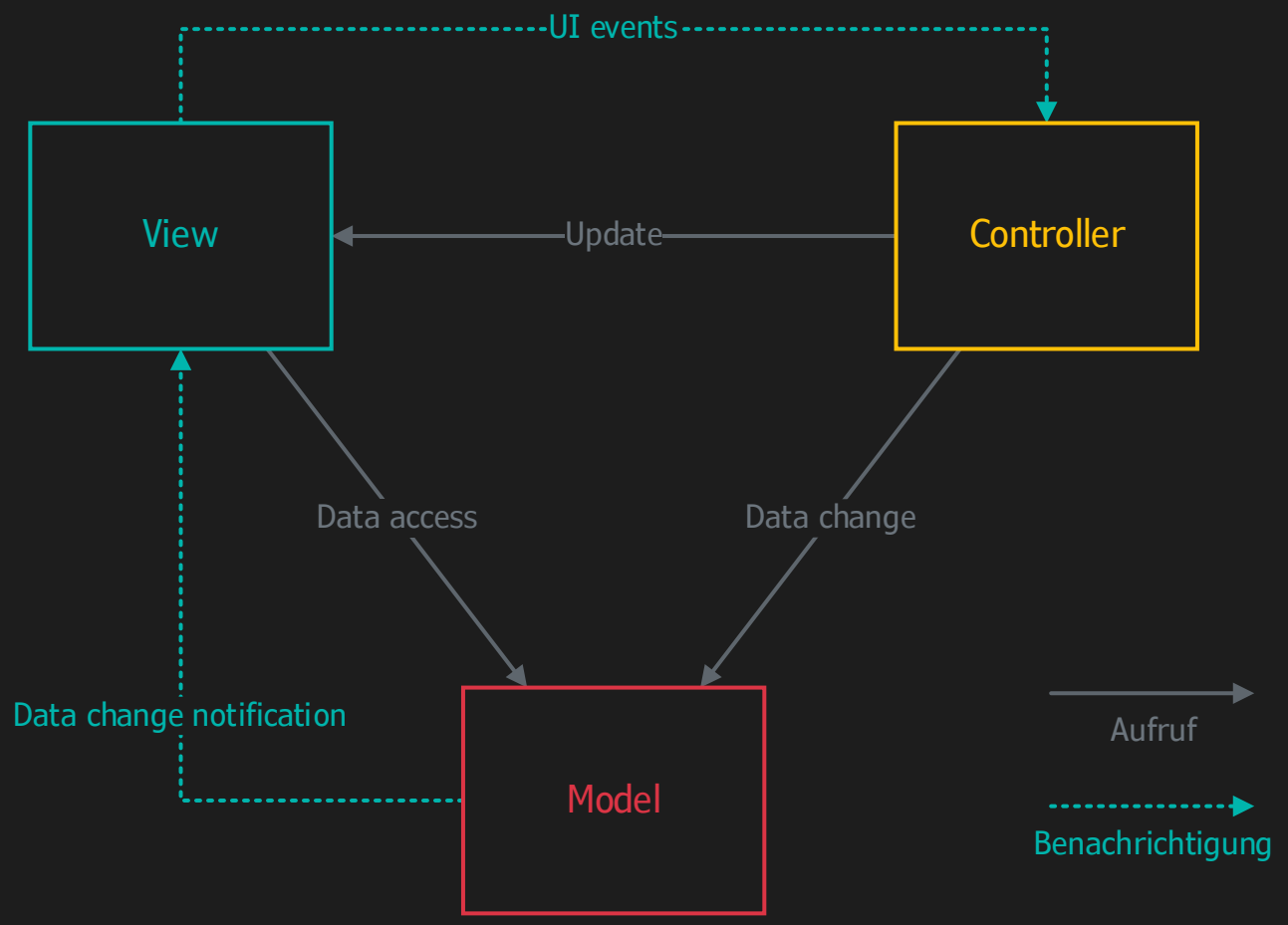
Nachteile

- Trennung oft nicht eindeutig
- Performance, da Anfragen ggf. durchgereicht werden
- Erschwerte Testbarkeit
- Änderungen können problematisch werden, wenn mehrere Schichten betroffen sind

Model-View-Controller (1)

- Model-View-Controller (MVC), 1979 für Smalltalk beschrieben
- Model
  - Enthält fachliche Daten und Funktionen
  - Unabhängig von GUI-Technologie
- Controller
  - Steuerung der Interkation mit Nutzenden
  - Weitergabe von Eingaben an Model & View
  - Zu jedem Controller gehört genau ein Model
- View
  - Anzeige der Model-Daten in der GUI
  - Zu jedem View gehört genau ein Controller (und damit auch genau ein Model)
  - Ein Controller kann mehrere Views steuern
- Ereignisse (Events), um Konsistenz sicherzustellen (Change Propagation)
- Interaktive GUI-Systeme, z.B. Webanwendungen mit JavaScript-Frameworks, z.B. Angular (Google)

Model-View-Controller (2)



- Beispielablauf bei UI-Eingabe: View meldet »Eingabeereignis« an Controller → Controller führt Änderung in Model durch → Model benachrichtigt View über Änderung → View holt aktuelle Daten aus Model und zeigt diese an

Model-View-Controller (3)

Vorteile

- Trennung von Verantwortlichkeiten [→ Seperation of Concerns]
- Mehrere Views pro Model möglich, auch in unterschiedlichen Technologien
- Bereits in vielen Frameworks enthalten

Nachteile

- Implementierung aufwendig
- Debugging komplex
- Performanz: Benachrichtigung von Modellen mit mehreren Controller-View-Paaren zeitaufwendig

Event-Driven Architecture (EDA)

- Kommunikation durch Ereignisse (Events)
  - Von Sender (Event Source) zu Empfänger (Event Sink)
  - Prinzip »Tell, don’t ask«
  - Vorteil: lose Kopplung gegenüber direkten Operationsaufrufen, z.B. Remote Procedure Call (RPC)
- Varianten
  - Broadcasting bzw. Publisher/Subscriper: Empfänger (Subscriber) registriert sich bei Sender (Publisher); Sender übermittelt Ereignisse direkt an alle Empfänger
  - Sender übermittelt Ereignisse an Puffer (Message Queue); Empfänger holt diese dort mit oder ohne Empfangsbestätigung ab (Fire & Forget)
- Message-Service-Architektur, z.B.
  - Routing, Reihenfolge (z.B. Priority Queues), Vorverarbeitung (z.B. Verschlüsselung), Zusammenfassung oder Aufteilung von Nachrichten
  - Zuverlässige Zustellung (Reliable Messaging), Protokollierung
  - Beispiele: Enterprise Service Bus (ESB), Message-Oriented Middleware (MOM)

Service-Oriented Architecture (SOA)

- Übergreifendes Thema, insbesondere Enterprise Systems
- Schichtenbasierter Aufbau des Gesamtsystems
- Bereitstellung sämtlicher Funktionalitäten über interoperable, lose gekoppelte und technologieneutrale Services
  - Selbstständig betriebene, unabhängige Softwareeinheiten
  - Realisieren Geschäftsfunktionen, z.B. Kundenverwaltung
  - Gemeinsame Datenhaltung
  - Entfernte, unabhängige Aufrufe gemäß öffentlichen Schnittstellen (Remote Invocation)
  - Bereitstellung der Services durch Enterprise Service Bus (ESB)
  - Services werden von verschiedenen UIs & Drittsystemen genutzt
  - Services von Drittsystemen können integriert werden
- Service-Vertrag legt Merkmale fest, z.B.
  - Service-Adresse (Service Endpoint)
  - Operationen & Daten
  - Policies zu Sicherheit, Verfügbarkeit, Kosten etc.

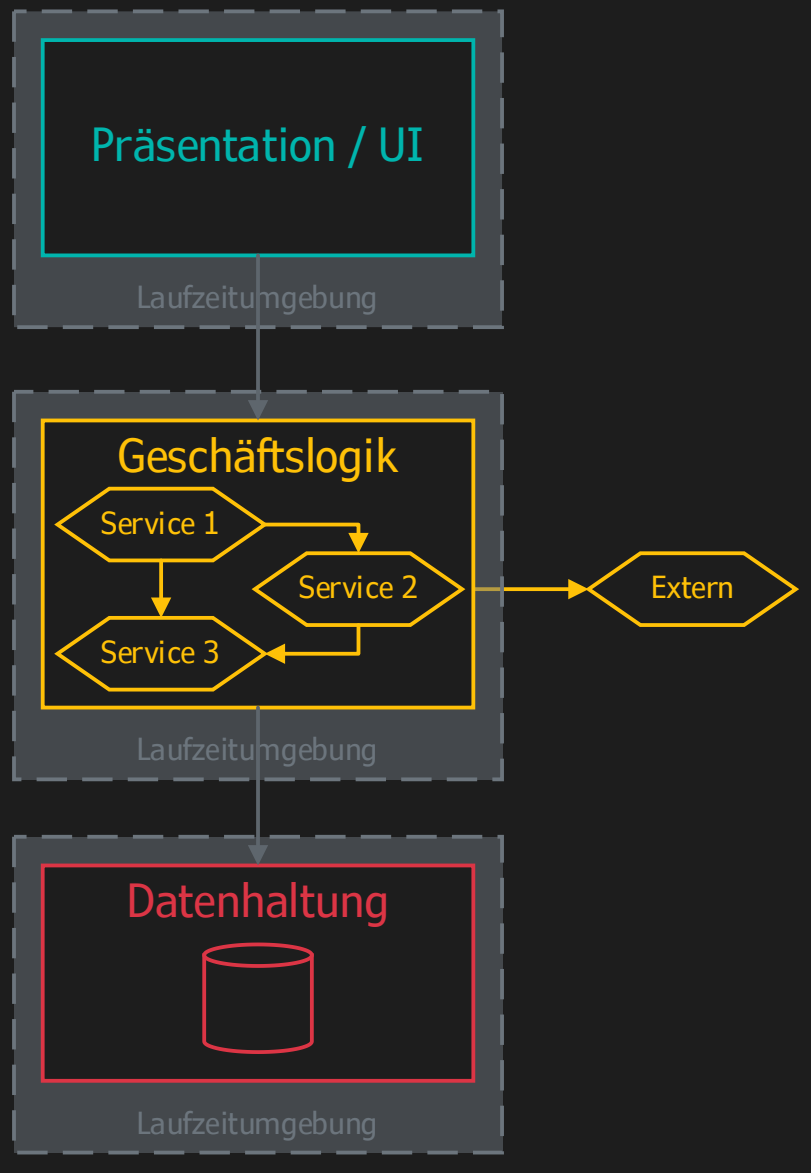
Resource-Oriented Architecture (ROA)

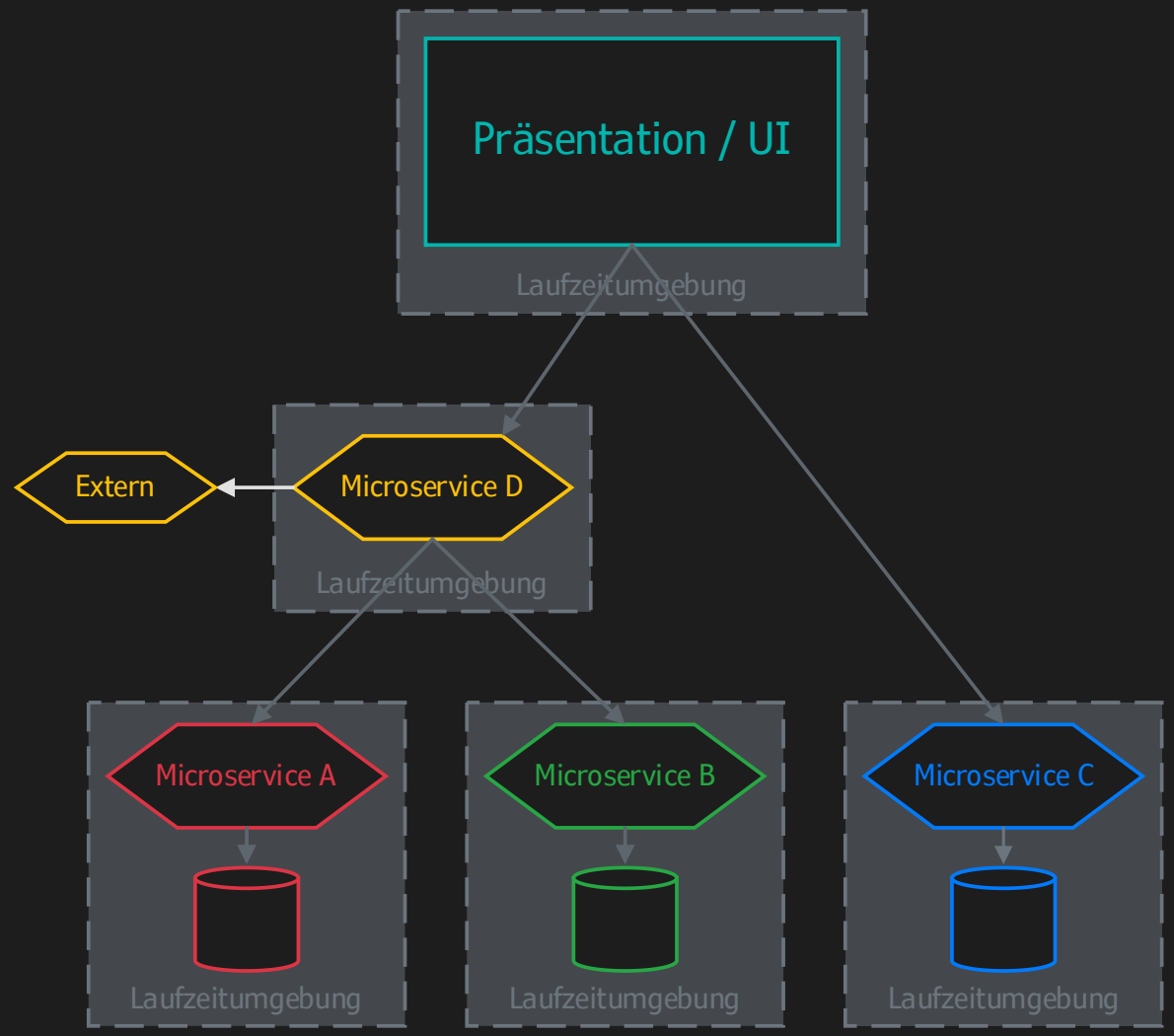
- Basieren auf Representational State Transfer (REST)
  - Kein Protokoll, sondern Paradigma
  - Abstraktion des WWW, geringerer Overhead
  - Zustandslos (stateless): REST-Nachricht muss alle Informationen enthalten, die für Verarbeitung notwendig sind (bessere Skalierbarkeit)
  - Adressierbarkeit von Ressourcen über Uniform Resource Identifier (URI)
  - Hypermedia as the Engine of Application State (HATEOAS)
  - Darstellungsform nicht festgelegt, kann z.B. HTML, XML oder JSON sein
  - Authentifizierung muss separat erfolgen, z.B. über OAuth
  - Verschlüsselung muss separat erfolgen, z.B. über HTTPS
- Typisches ROA-Austauschformat: JavaScript Object Notation (JSON)
  - Kompakter und einfacher lesbar, geringer Overhead
  - Valides JavaScript, Interpretation durch eval() möglich [Achtung: mögliche Sicherheitslücke]
  - Nur Datenaustauschformat
  - Nicht struktur-beschreibend, nur Syntax-Konvention
  - Eher ungeeignet für restriktive Schnittstellen

Microservices Architecture (MSA)

- Ziel: stärkere Modularisierung & Flexibilität, insbesondere für CI/CD, DevOps, FDD
- Microservice
  - Isolierter Service, eigene Laufzeitumgebung, autonome Datenhaltung
  - Nur eine einzige Aufgabe, die wird aber besonders gut erbracht (»Do one thing and do it well«)
  - Hohe fachliche Kohäsion, geringe Abhängigkeit von anderen Services
  - Interdisziplinäre Teams, vollständige Verantwortlichkeit für einen Microservice (»You build it, you run it«) [→ Amazon]
- System of Systems
  - Überwiegend aus Microservices zusammengesetztes Gesamtsystem
  - Orchestrierung dezentral, ohne schwerfällige Middleware (»Smart endpoints, dumb pipes«)
- Cloud: Function as a Service (FaaS), skalierbar durch Serverless Computing, z.B. Azure Functions, AWS Lambda

SOA vs MSA


Präsentation / UI
Laufzeitumgebung
Geschäftslogik
Service 1
Service 2
Service 3
Extern
Laufzeitumgebung
Datenhaltung
Laufzeitumgebung


Präsentation / UI
Laufzeitumgebung
Extern
Microservice D
Laufzeitumgebung
Microservice A
Laufzeitumgebung
Microservice B
Laufzeitumgebung
Microservice C
Laufzeitumgebung